

AgentBench on a Linux VM

Setup guide · 19 September 2026

Run AgentBench's two daemons on a Linux VM, then drive that machine's projects from the AgentBench desktop app on your own laptop. Nothing graphical is installed on the VM, and the terminals you open run on the VM rather than locally.

This guide has not been tested end to end on a fresh VM.

Expect to do some debugging, and tell whoever wrote it what you hit.

1. Before you start

You need a Linux VM you can SSH into as a normal user, plus AgentBench already installed on your own laptop.

What	Why
A Linux VM (Ubuntu 22.04+ or similar)	Where the agents and terminals actually run
SSH access as a non-root user	The daemons run as that user and own its projects
Rust toolchain, from rustup.rs	Builds the two daemons
Node.js 20 or newer	Builds the web assets the gateway serves
git, curl, build-essential	curl is not optional — see section 6
Claude Code, as the same user	The agents themselves

No desktop environment, no X server and no webkit are needed. The daemons build without them.

```
sudo apt-get update && sudo apt-get install -y git curl build-essential
```

2. Build and install

Build on the VM itself. Cross-compiling from a Mac is not set up.

The `--no-default-features` flag is what keeps this small: it drops the desktop app and everything it drags in, leaving only the two daemons.

```
git clone https://github.com/moodydoesdev/AgentBench.git
cd AgentBench

# the two daemons - no GUI toolkit pulled in
cd src-tauri
cargo build --release --no-default-features \
  --bin agentbench-broker --bin agentbench-gateway
cd ..

# the web assets the gateway serves
npm ci
npm run build
```

The Rust build takes 5 to 15 minutes on a small VM the first time.

Then put everything where the service files expect it:

```
sudo install -m755 src-tauri/target/release/agentbench-broker /usr/local/bin/
sudo install -m755 src-tauri/target/release/agentbench-gateway /usr/local/bin/

sudo mkdir -p /usr/local/share/agentbench
sudo cp -r dist /usr/local/share/agentbench/

sudo cp packaging/systemd/agentbench-*.service /etc/systemd/system/
sudo systemctl daemon-reload
```

3. Start the services

The units are templates, so the username goes after the @. Replace youruser with the account that owns the projects.

```
sudo systemctl enable --now agentbench-broker@youruser
sudo systemctl enable --now agentbench-gateway@youruser
```

Two processes, different jobs:

Process	Listens on	Job
agentbench-broker	127.0.0.1:8317, loopback only	Owns the ptys. No authentication, so it must never leave loopback.
agentbench-gateway	0.0.0.0:8473	Token-authenticated bridge. The only part anything outside the VM talks to.

If node is installed somewhere unusual (nvm, asdf, a custom prefix), edit PATH= in /etc/systemd/system/agentbench-broker@.service so claude is findable, then run `sudo systemctl daemon-reload` and restart. A systemd service starts from a much thinner environment

than a login shell, so anything on your interactive PATH is not automatically on the service's.

4. Connect from your laptop

The gateway speaks plain HTTP with no TLS, so it must never be exposed directly to the internet. Pick one of the two options below.

Option A - Tailscale (easier)

On the VM:

```
tailscale up
tailscale serve --bg 8473    # optional, gives you HTTPS
agentbench-gateway pair
```

That prints an address and a code like 4821-9937, good for 10 minutes and one use.

In the desktop app: Settings > Linked benches, enter the address and code it printed, then press Link both ways.

Option B - SSH tunnel

On your laptop:

```
ssh -fN -L 8473:localhost:8473 youruser@yourvm
```

On the VM, run `agentbench-gateway pair` for a code.

In the desktop app: Settings > Linked benches, the code from above, and for the address type `http://localhost:8473` — NOT the address that pair printed. Over a tunnel your end is localhost; the address pair shows is the VM's own, which your laptop cannot reach.

Expect the link to report that the reverse direction failed.

That is normal and harmless. An `ssh -L` tunnel only carries traffic one way, so the VM cannot call back. The direction you need already succeeded.

Keep the tunnel up, because it is the connection. In `~/ .ssh/config`:

```
Host yourvm
  HostName 10.0.0.5
  User youruser
  LocalForward 8473 localhost:8473
  ServerAliveInterval 30
  ExitOnForwardFailure yes
```

Then just `ssh -fN yourvm`. If it drops, the app reconnects on its own once the tunnel is back, with no need to link again.

5. Check it worked

On the VM:

```
agentbench-gateway status
```

Look for "brokerConnected": true. If it says false, the gateway is running but cannot find the broker — check `systemctl status agentbench-broker@youruser`.

In the app, the VM appears as its own section in the left sidebar, below your local projects, with a dot that fills in when it is connected. Click one of its projects and the panes you open there run on the VM.

Two things to try, to confirm both halves work:

- New agent in a VM project — confirms the broker is spawning ptys
- `Ctrl+T` for the terminal dock — opens a shell on the VM, in the tab strip along the bottom

Logs, if you need them:

```
journalctl -u agentbench-broker@youruser -f
```

6. If something breaks

Symptom	Cause	Fix
Panes look dead, agents never respond	curl is missing. It carries Claude Code's hook events, and without it nothing reports back, silently.	Install curl, then restart the broker
Broker exits at startup	HOME is not set in the service environment	The shipped unit sets it. If you wrote your own, add <code>Environment=HOME=/home/youruser</code>
Agent starts but claude is not found	The service PATH is thinner than your login shell's	Add node's bin directory to <code>PATH=</code> in the broker unit
App shows the bench but no projects	Nothing has been opened on the VM yet	Projects come from the VM's own AgentBench state. Open one there first.
VM shows up as "workstation"	HOSTNAME is not exported by systemd	The shipped unit sets it. Reload if you replaced the file.

Lock the port down

The gateway always binds `0.0.0.0:8473` and has no option to restrict that yet. If the VM has a public IP, firewall the port. With Tailscale or an SSH tunnel you never need it reachable directly.

```
sudo ufw allow from 100.64.0.0/10 to any port 8473 # tailnet only
sudo ufw deny 8473
```

This matters more than it looks.

A paired device token is full code execution on the VM as that user, and the agents run permission-skipped.

Stopping

```
sudo systemctl stop agentbench-gateway@youruser agentbench-broker@youruser
```

The broker kills its panes and cleans up after itself on stop, so nothing is left holding a port.

Known limits

- No TLS of its own. Tailscale or SSH provides it.
- Memory-pressure hibernation is weaker on Linux than on macOS. Leave it off unless you need it.
- Dictation is macOS-only and is not part of these builds.